TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
ĐHQG-HCM

BÁO CÁO LAB 01

BLOCKCHAIN LAYER-1 – MÔ PHỎNG ĐỒNG THUẬN BFT NHIỀU NODE

Môn học: Blockchain

Sinh viên thực hiện: Võ Lê Ngọc Hiếu 23127366
Kiều Duy Hiếu 23127365
Cao Lê Gia Phú 23127535
Võ Thành Nhân 23127533

TP. Hồ Chí Minh, tháng 11 năm 2026

1. THÔNG TIN THÀNH VIÊN

Nhóm có 4 thành viên. Mỗi người phụ trách một tầng của hệ thống và các kịch bản kiểm thử gắn với tầng đó; mọi thay đổi đi qua pull request và được thành viên khác review.

MSSV	Họ tên	GitHub	Phụ trách
23127366	Võ Lê Ngọc Hiếu	NgocHieu162	A – Nền tảng: models, encoding, chữ ký, logger, tích hợp node; kịch bản T1, T3
23127365	Kiều Duy Hiếu	DuyHieu251005	B – Execution (state, mempool, kiểm tra tx), fault injection, kiểm tra safety/tái lập; T4, T5, T8
23127535	Cao Lê Gia Phú	clgp-aint-cool	C – TCP transport, HELLO, static peers, gossip, header/body; T2
23127533	Võ Thành Nhân	vtnHAN-SOLO	D – Consensus engine, VoteSet, locking, timeout, hành vi Byzantine; T6, T7

2. CẤU TRÚC DỰ ÁN

Bài nộp theo đúng cấu trúc `Lab01_<MSSV>/` của đề. Mã nguồn là gói Python `l1sim`; mỗi thư mục con ứng với một tầng của kiến trúc ở mục 4.

Đường dẫn	Nội dung
`src/l1sim/`	`models` (Tx, BlockHeader, Block, Vote, Hello, Message), `encoding` (JSON chuẩn tắc, hash), `crypto` (Ed25519), `config`, `logger`, `node`, CLI `run_node`, `launch_local`, `client`
`consensus/`	`engine` (máy trạng thái round), `vote_set` (đếm phiếu, quorum, equivocation), `byzantine` (hành vi lỗi cho T3/T6/T7)
`execution/`	`state` (key–value, nonce, state hash), `mempool`, `validation` (kiểm tra tx và block)
`network/`	`tcp_transport`, `hello`, `static_peers`, `gossip`, `faults` (FaultInjector), `peer_limits`
`verification/`	`check_safety`, `check_reproducible`
`tests/`	`run_tests.py` (lệnh chạy duy nhất), `unit/` (28 file), `e2e/` (T1–T8), `integration/`
`config/`	`genesis.json` (10 validator, public key) và 7 scenario `t1_normal` … `t7_byzantine`
`logs/`	`events/<scenario>/V*.jsonl`, `results/<scenario>/V*.json`, `verdict.json`, stdout/stderr từng node
`dashboard/`	giao diện web để demo (`server.py`, `index.html`); chỉ dùng khi trình bày, không nằm trong file zip nộp
`README.md`, `REPORT.pdf`	hướng dẫn chạy, đọc log, tái lập; báo cáo này

3. GIỚI THIỆU

3.1. Mục đích hệ thống

Hệ thống mô phỏng một blockchain Layer-1, trong đó mỗi validator là một **process riêng** giao tiếp qua **TCP thật**. Các validator cùng duy trì một state dạng key–value, nhận giao dịch có chữ ký, gom thành block và đồng thuận theo ba bước Propose → Prevote → Precommit. Mục tiêu là chứng minh bằng thực nghiệm hai tính chất của đề:

- **Safety** luôn đúng: hai node trung thực không bao giờ finalize hai block khác nhau ở cùng height, kể cả khi mạng mất, trễ, trùng, đảo thứ tự message và có tối đa f validator Byzantine.
- **Liveness** sau GST: khi mạng ổn định, chain tiếp tục tiến.

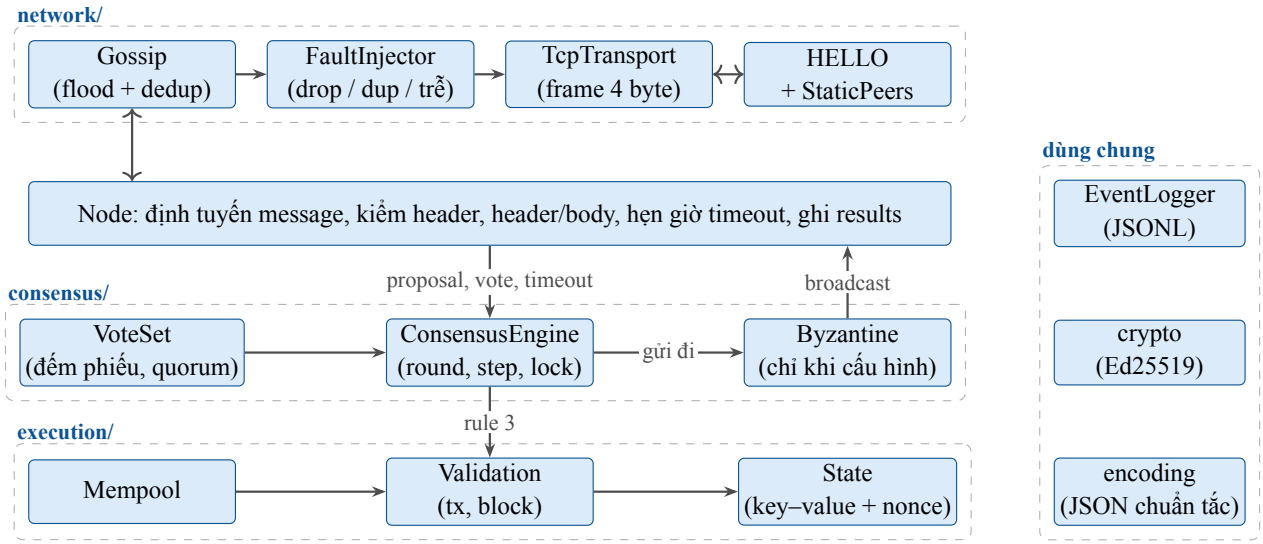
Cấu hình chuẩn có $n = 10$ validator, nên $f = \lfloor (n-1)/3 \rfloor = 3$ và quorum $= n - f = 2f + 1 = 7$.

3.2. Các lựa chọn thiết kế chính

- **Python 3.10 + asyncio**: mỗi node chạy một event loop; chỉ phụ thuộc thư viện `cryptography` cho Ed25519.
- **Mã hóa chuẩn tắc**: mọi giá trị được ký hoặc hash đều qua cùng một hàm encode (JSON, khóa sắp xếp, không khoảng trắng, UTF-8, cấm số thực). Bộ giải mã từ chối message không chuẩn tắc, nên mỗi giá trị chỉ có đúng một dạng bytes.
- **Tách domain**: chữ ký và hash có tiền tố `<DOMAIN>:lab01` (TX, HEADER, VOTE, HELLO, BLOCK, STATE…), nên một chữ ký hợp lệ không thể dùng lại ở ngữ cảnh khác hay chain khác.
- **Đồng thuận kiểu Tendermint** có locking, timeout tăng dần theo round và round-skip. Engine là máy trạng thái thuần: không chạm mạng, đồng hồ hay `asyncio`, mọi tác dụng phụ đi qua callback, nên unit test được từng luật.
- **Mạng full mesh tĩnh** (static peers) + gossip flood có dedup; block truyền theo kiểu **header trước, body sau**.
- **Tất định**: khóa sinh từ seed, lỗi mạng là hàm hash của `(seed, src, dst, message_id)`, hành vi Byzantine suy ra từ message trung thực, results chỉ chứa dữ liệu đã sắp xếp. Nhờ đó cùng seed cho results giống từng byte (T8).

4. THIẾT KẾ HỆ THỐNG

4.1. Sơ đồ kiến trúc



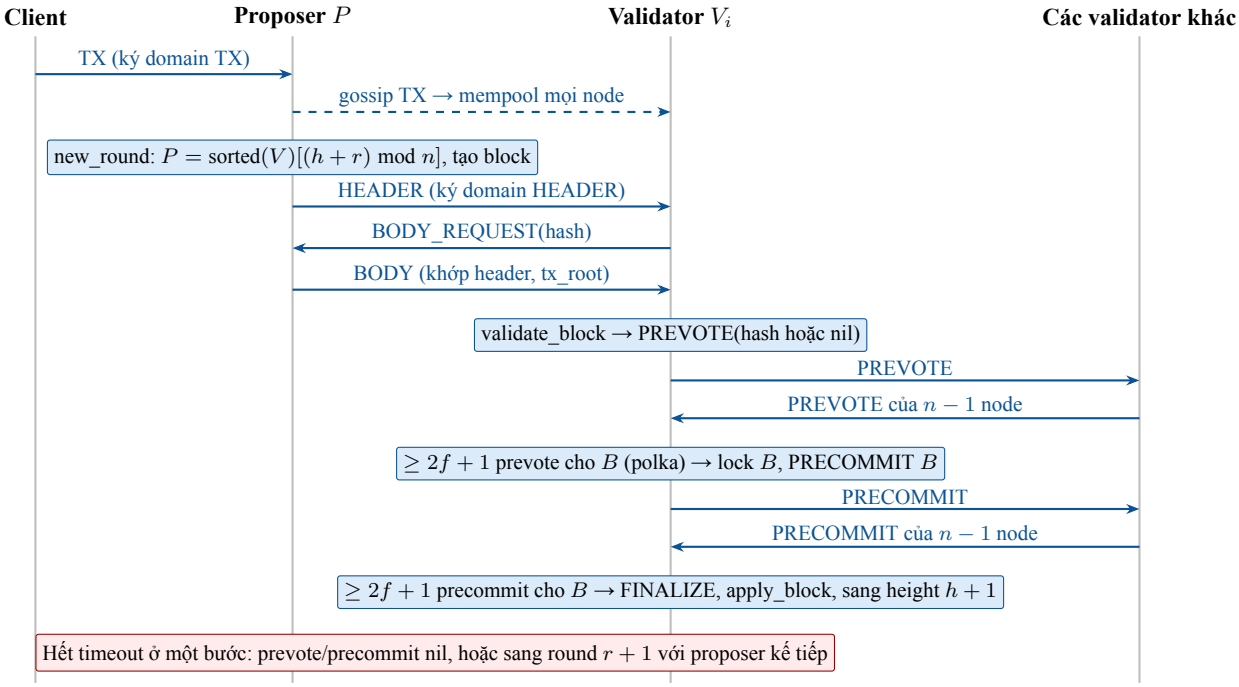
Hình 1: Các module trong một validator. Mỗi process chạy đúng một bộ như trên; 10 process nối full mesh qua TCP.

4.2. Vai trò của các module

- **TcpTransport** chỉ chuyển bytes: mỗi frame là 4 byte độ dài big-endian + nội dung, tối đa 1 MB. Mỗi peer có một coroutine đọc; ghi một frame là một lệnh `write` nên frame không xen nhau. Peer đọc quá chậm (buffer gửi > 4 MB) bị bỏ message thay vì làm node chậm theo.
- **HELLO** xác thực peer ngay sau khi kết nối: mỗi bên gửi `Hello` (chain_id, node_id, địa chỉ, height đã finalize) ký bằng domain `HELLO`. Bên nhận kiểm theo thứ tự `malformed` → `wrong_chain` → `unknown_peer` → `self_connect` → `bad_signature`.
- **StaticPeers** dựng full mesh từ `network.peers`: node đứng sau trong thứ tự sắp xếp chủ động nối tới node đứng trước, nên mỗi cặp chỉ có một kết nối; mất kết nối thì quay số lại sau 0,1 s.
- **Gossip** phân hai loại message: FLOOD (TX, HEADER, VOTE) gửi cho mọi peer và chuyển tiếp nếu tầng trên chấp nhận; DIRECT (BODY_REQUEST, BODY) gửi 1–1.

- **FaultInjector** nằm giữa Gossip và transport, quyết định drop / gửi / nhân đôi và độ trễ cho từng message trên từng link.
- **Node** nối mọi thứ: kiểm tra sơ bộ (height, proposer, chữ ký header), quản lý header/body, hẹn giờ timeout cho engine, đưa tx vào mempool và ghi results khi đạt `max_height`.
- **ConsensusEngine** chạy luật vote; **VoteSet** là nơi duy nhất đếm phiếu; **Byzantine** (nếu có trong scenario) biến đổi message gửi đi của một node, còn engine bên trong vẫn chạy luật trung thực.
- **Execution**: `validate_tx` (chữ ký → quyền sở hữu key → nonce), `validate_block` (proposer, parent, chữ ký header, tx_root, chạy thử tx, state hash), `apply_block`.

4.3. Luồng message trong một height



Hình 2: Một height ở trường hợp bình thường. Mọi vote đều được gossip flood; mũi tên chỉ vẽ luồng chính.

4.4. Hoạt động chi tiết và các tương tác chính

Mô hình dữ liệu. Tx gồm `(sender, key, value, nonce, signature)`; `key` phải có dạng `<sender>/...` (người gửi chỉ ghi vào vùng của mình). Header gồm `height, round, parent_hash, tx_root, state_hash, proposer, signature`; block hash là SHA-256 của header bỏ chữ ký, domain `BLOCK`. `tx_root` là hash (domain `TXS`) của danh sách tx đã mã hóa chuẩn tắc. `state_hash` là hash (domain `STATE`) của toàn bộ map dữ liệu và map nonce. Vote gồm `(phase, height, round, block_hash | null, voter, signature)`. Mỗi message trên dây là `Message(kind, origin, height, payload)`.

Chọn proposer. Mọi node tính $P(h, r) = \text{sorted}(V)[(h + r) \bmod n]$ cục bộ, không cần trao đổi. Round tăng thì proposer xoay vòng, nên proposer im lặng chỉ làm mất một round.

Một round. Khi vào round, proposer lấy tối đa `max_txs_per_block` tx hợp lệ từ mempool (đúng nonce kế tiếp), chạy thử trên bản sao state để tính `state_hash`, ký header và broadcast. Node khác nhận HEADER, kiểm chữ ký và proposer rồi xin BODY từ chính peer đã gửi header; nếu sau 300 ms chưa có body thì hỏi mọi validator khác. Khi có block, engine quyết định PREVOTE theo luật lock (mục 5.3); gặp polka thì PRECOMMIT; gặp $2f + 1$ PRECOMMIT cho một block đã có thì finalize. Mỗi bước có timeout $T_{\text{step}}(r) = T_{\text{base}} + r \cdot \delta$ (1000 ms + r· 300 ms).

Chain và view. Mỗi node giữ chain đã finalize, state hiện tại và `CommitCertificate` ($2f + 1$ precommit). Khi đạt `max_height`, node ghi `results/V*.json` (height, block hash, proposer, round, commit_round, tx id, state hash). `check_safety` đọc results của mọi node và kiểm: không có hai block

khác nhau ở cùng height, cùng state hash ở height chung, đủ quorum node đạt `max_height`.

5. GHI CHÚ CÀI ĐẶT

5.1. Các lớp bảo vệ (guard)

Mỗi message phải qua lần lượt các guard dưới đây; bị loại ở bước nào thì node ghi event kèm `reason` tương ứng và **không chuyển tiếp** message đó.

Guard	Cách hiện thực	Event / reason
Kích thước, định dạng	Frame > 1 MB bị bỏ; `decode` kiểm đúng tập trường, kiểu, và encode lại phải trùng bytes đã nhận	`gossip_malformed`
Xác thực peer	HELLO ký domain HELLO, peer phải thuộc genesis, không tự nối chính mình, đúng chain_id	`handshake_failed`
Nguồn gửi	Peer và `origin` phải là validator trong genesis; client chỉ được gửi TX	(bỏ qua)
Trùng lặp	Gossip dedup theo `message_id` (hash domain MSG) trước lệnh `await` đầu tiên; VoteSet dedup theo (voter, height, round, phase, giá trị)	`gossip_duplicate`, `duplicate_ignored`
Height cũ / ngoài phạm vi	Message có height < height hiện tại hoặc > `max_height` bị bỏ	`ignored_old_height`
Chữ ký vote	Kiểm Ed25519 với domain VOTE; nếu sai thì thử các domain khác để phân biệt hai lỗi	`vote_rejected`: `bad_signature`, `wrong_domain`
Header	Chữ ký domain HEADER, proposer phải đúng $P(h, r)$	`header_rejected`
Body	Header của body phải trùng header đã nhận; `tx_root` khớp danh sách tx	`body_rejected`
Block (rule 3)	Đúng proposer, đúng parent, chữ ký, tx_root, từng tx hợp lệ khi chạy tuần tự, state hash sau thực thi khớp header	`invalid_block`
Giao dịch	Chữ ký domain TX → quyền sở hữu key → nonce; mempool nhận nonce tương lai nhưng block chỉ nhận đúng nonce kế tiếp; trùng (sender, nonce) bị từ chối	`bad_signature`, `not_owner`, `bad_nonce`
Tự ký hai lần	Engine không bao giờ ký hai vote khác nhau cho cùng (height, round, phase)	`refuse_double_sign`
Equivocation	VoteSet lưu bằng chứng khi một voter ký hai giá trị cho cùng slot	`equivocation_detected`

Rate limit. Để yêu cầu giới hạn tốc độ và chặn peer. Nhóm có module `PeerLimits` (giới hạn 1000 message/giây mỗi peer cả chiều vào và ra, chặn peer vượt ngưỡng trong 10 s, có unit test), cùng ngưỡng buffer 4 MB ở transport. Tuy nhiên ở bản nộp, `PeerLimits` **chưa được gắn vào đường nhận của node**; đây là hạn chế được nêu lại ở mục 6.3.

5.2. Chiến lược gossip và header trước body

Mỗi node nối trực tiếp với mọi validator khác. Khi tạo message FLOOD, node đánh dấu `message_id` là đã thấy rồi gửi cho mọi peer. Node nhận chạy handler; chỉ khi handler chấp nhận (`True`) thì mới chuyển tiếp **đúng bytes đã nhận** cho mọi peer trừ nơi gửi. Vì đánh dấu đã thấy trước khi xử lý, một message bị từ chối sẽ không được xét lại, nên handler chỉ trả `False` cho lỗi vĩnh viễn (chữ ký sai, height cũ, nonce cũ). Mỗi message được xử lý đúng một lần dù tới nhiều bản: ở T1, 10 node ghi khoảng 6000 `gossip_duplicate` và không bản nào được xử lý lại.

Block không được flood nguyên khối: node gửi `BLOCK` nội bộ, tầng gửi tách thành HEADER (flood). Node nhận header hợp lệ mà chưa có body thì gửi `BODY_REQUEST` tới peer vừa gửi header, sau 300 ms hỏi thêm mọi validator khác. Node đã có header nhưng chưa có body sẽ ghi nhớ người hỏi và trả body khi nhận được. Body của 20 height gần nhất được giữ lại để trả lời node chậm.

Mô phỏng lỗi mạng. `FaultInjector` băm `seed|src|dst|message_id` bằng SHA-256 rồi lấy các đoạn của digest làm số ngẫu nhiên: $u <$ `drop_rate` thì drop (chỉ khi height < `gst_height`); u nằm trong khoảng tiếp theo có độ rộng `duplicate_rate` thì gửi hai bản. Độ trễ có hai cực: ngắn (1 đến 10% timeout, đủ để đảo thứ tự) hoặc, trước GST, dài (hơn hai lần timeout lớn nhất). Thiết kế hai cực tránh vùng trễ xấp xỉ timeout, nơi kết quả phụ thuộc vào thời điểm chạy và phá T8. Quyết định chỉ phụ thuộc nội dung message, không phụ thuộc thứ tự gọi, nên mọi lần chạy cùng seed có cùng số phận cho cùng message.

5.3. Các bước đồng thuận và luật vote

Engine có một vòng `_progress` duy nhất, chạy sau mỗi input (proposal, vote, timeout) và áp lần lượt mọi luật có điều kiện đúng cho tới khi không còn gì thay đổi. Bảy luật của đề được hiện thực như sau:

1. **Mỗi (height, round, phase) chỉ một vote**: hàm ký vote của engine từ chối ký giá trị thứ hai (`refuse_double_sign`).
2. **Chỉ finalize với $2f + 1$ precommit cho cùng block**: `_try_finalize` xét mọi round của height hiện tại, kể cả round cũ, để node đã sang round sau vẫn finalize khi precommit muộn tới. Nếu có quorum cho block chưa nhận được, node chờ (`commit_waiting_for_block`).
3. **Chỉ prevote block hợp lệ**: `validate_block`; block sai thì prevote nil.
4. **Lock**: khi precommit B vì thấy polka cho B ở round r, node lưu `locked_block = B`, `locked_round = r` (event `lock`).
5. **Đã lock thì prevote block đã lock**, trừ khi thấy polka cho block khác ở round mới hơn `locked_round`. Proposer đang lock thì đề xuất lại đúng block đó (`repropose`, cùng header, cùng hash).
6. **Timeout**: hết PROPOSE thì prevote nil; hết PREVOTE thì precommit nil; hết PRECOMMIT thì sang round $r + 1$. Ngoài ra có round-skip: thấy $f + 1$ voter khác nhau ở round cao hơn (chắc chắn có node trung thực) thì nhảy tới round đó.
7. **Không đếm vote trùng**: VoteSet đếm mỗi voter tối đa một lần cho mỗi giá trị trong một slot; quorum là $n - f$ voter khác nhau.

Vì sao an toàn. Hai quorum kích thước $2f + 1$ trong $n = 3f + 1$ validator giao nhau ít nhất $f + 1$ voter, trong đó có ít nhất một node trung thực; node đó không ký hai giá trị cho cùng slot, nên hai block không thể cùng đạt quorum ở một round. Giữa các round, $f + 1$ node trung thực đã lock block được finalize sẽ chỉ prevote block đó, nên không block nào khác đạt polka ở round sau.

Polka evidence. Mỗi khi vào round mới, node đang theo một polka cũ gửi lại $2f + 1$ prevote có chữ ký của polka đó. Node bỏ lỡ polka kiểm chữ ký từng phiếu, thấy polka và prevote cùng block. Cơ chế này chỉ gửi lại vote thật nên không đổi luật nào (xem lỗi D6).

Byzantine. Node Byzantine chạy engine trung thực, chỉ có message gửi đi bị biến đổi: `silent` (không gửi), `equivocate` (thêm một block/vote mâu thuẫn, ký hợp lệ), `bad_signature` (đổi một ký tự hex của chữ ký), `wrong_domain` (ký lại bằng domain khác). Mọi biến đổi suy ra từ message gốc, không dùng số ngẫu nhiên, nên vẫn tái lập được.

5.4. Các lỗi khó và cách khắc phục

Các lỗi dưới đây chỉ lộ ra khi chạy nhiều process với lỗi mạng hoặc Byzantine; chi tiết đầy đủ nằm trong `docs/BUGS.md`.

1. **Mất safety khi chưa có locking.** Dựng lại phản ví dụ: V0 finalize B ở round 0, các node khác mất precommit round 0 rồi finalize C ở round 1. Nguyên nhân là bản đầu chưa có luật 4–5. Thêm locking; test `test_locking_keeps_safety_when_round_0_precommits_are_lost`.
2. **Locking theo đúng flowchart làm chain đứng.** Node đã lock prevote nil khi proposal là block khác, còn proposer luôn tạo block mới, nên nhóm đã lock và nhóm chưa lock không bên nào đủ quorum. Sửa: node lock prevote *block đã lock*; proposer đang lock đề xuất lại block đó; node chưa lock nhưng biết polka cũ thì prevote block của polka. Safety không đổi vì node lock vẫn không prevote block khác.
3. **Polka chỉ tới một phần mạng thì kẹt vĩnh viễn.** Round 0 chỉ V0–V3 thấy polka và lock B; V4–V9 không bao giờ biết B từng có polka vì gossip không gửi lại prevote cũ. Sửa bằng polka evidence (mục 5.3); có

test đối chứng cho thấy không có cơ chế này thì kẹt.

4. **T7 thỉnh thoảng kẹt vì không đếm vote thật của node equivocate.** VoteSet ban đầu chỉ giữ vote tới trước của mỗi voter. Một node nhận precommit giả của V7–V9 trước, nên precommit thật cho block được commit bị coi là equivocation và không đếm; node đó thiếu quorum mãi mãi. Sửa: đếm mỗi vote cho đúng giá trị của nó, mỗi voter tối đa một lần mỗi giá trị, vẫn lưu bằng chứng; vote xung đột mới được chuyển tiếp đúng một lần. Vẫn an toàn vì hai block cùng đạt quorum đòi hỏi ít nhất một node trung thực equivocate.
5. **Block xung đột bị vứt nên không finalize được.** Proposer Byzantine gửi hai block cho cùng (height, round); node nhận block thứ hai sau bỏ hẳn nó. Nếu mạng commit đúng block thứ hai, node có đủ precommit mà không có block (`commit_waiting_for_block` mãi). Sửa: block xung đột không chiếm slot proposal (không được prevote) nhưng vẫn được lưu theo hash.
6. **T7 chạy hai lần cho kết quả khác nhau.** Thứ tự hai proposal xung đột tới mỗi node phụ thuộc gossip, và số cặp equivocation mỗi node bắt được phụ thuộc thời điểm. Sửa: bản mâu thuẫn được gửi trễ gấp đôi timeout dài nhất; results chỉ ghi danh sách `equivocators` đã sắp xếp, chi tiết từng cặp để trong event log.
7. **Engine crash ở event đầu tiên khi ghép với logger**: cả engine lẫn logger cùng ghi trường `node`. Sửa: logger sở hữu node id.

6. KIỂM THỬ VÀ KẾT QUẢ

6.1. Cách chạy

Một lệnh duy nhất chạy toàn bộ: `python tests/run_tests.py`; chọn kịch bản bằng `--scenario T1 … T8`, chỉ unit bằng `--unit`, chỉ E2E bằng `--e2e`. Mỗi test E2E khởi động 10 process `run_node` trên TCP loopback, chờ chạy xong rồi kiểm nội dung results, state, số giao dịch và event log, không chỉ dựa vào exit code. Unit test (28 file) kiểm riêng chữ ký và domain, encoding chuẩn tắc, nonce, ownership, state hash, quorum, locking, round-skip, polka evidence, framing, HELLO, gossip, quyết định của FaultInjector và rate limit.

Kết quả lần chạy đầy đủ (ngày 06/11/2026, macOS, Python 3.10): 365 test pass, 0 fail, tổng thời gian 113,1 s, exit code 0. T1 còn được chạy với 8, 9 và 10 validator, có và không có drop.

6.2. Kết quả từng kịch bản E2E

Số liệu "quan sát" lấy từ `logs/events` và `logs/results` của lần chạy mỗi scenario bằng launcher (cộng dồn 10 node).

Kịch bản	Kỳ vọng	Quan sát	KQ
T1 Bình thường	10 node finalize cùng 5 block, cùng state hash	5 height đều commit ở round 0, proposer V1→V5, mỗi block 4 tx (đủ 20 tx); 50 `finalize`, cùng state hash `6820bef7…` ở mọi node	**PASS**
T2 Trùng / đảo thứ tự (dup 0,3)	Không đếm vote hai lần, không thực thi tx hai lần, chain nhất quán	1960 `fault_duplicate`, 7682 `gossip_duplicate`; không vote nào bị đếm hai lần (unit VoteSet kiểm riêng); 20 tx id không lặp, 10 node cùng chain	**PASS**
T3 Chữ ký sai / sai domain	Mọi node trung thực từ chối hai vote lỗi với đúng reason, vẫn finalize	V5 gửi prevote sai chữ ký, V6 ký prevote bằng domain TX: 9 `vote_rejected/bad_signature` và 9 `wrong_domain` (đúng 9 node nhận mỗi loại); vẫn 5 block, cùng state hash với T1	**PASS**
T4 Replay tx	20 tx đầu vào với 11 cặp (sender, nonce) khác nhau; chỉ 11 tx được commit	Bản gửi lại y hệt bị gossip dedup; 9 tx khác nội dung nhưng trùng (sender, nonce) bị mempool từ chối và không được flood; `verdict.total_txs` = 11, tx id không lặp, safety giữ	**PASS**
T5 Mất / trễ trước GST (drop 0,3, dup 0,1, GST = 2)	Safety giữ; sau GST tiếp tục finalize tới `max_height`	993 `fault_drop`, 872 `fault_duplicate`, 24 `timeout`, 1 `round_skip`; height 1 commit ở round 1 (proposer V2), height 2–5 commit ở round 0; mọi node cùng chain	**PASS**
T6 Proposer im lặng	Proposer height 2 (V2) không gửi; timeout và round 1 với V3 finalize	`byzantine_action silent`; 9 prevote nil, timeout PROPOSE, round 1 proposer V3 finalize height 2 (`commit_round`=1); các height khác ở round 0	**PASS**

Kịch bản	Kỳ vọng	Quan sát	KQ
T7 $f = 3$ equivocate (V7–V9, 9 height)	Phát hiện equivocation, node trung thực không finalize mâu thuẫn	57 `byzantine_action`, 322 `equivocation_detected`; 9 height đều commit, 7 node trung thực cùng chain; results ghi `equivocators = [V7, V8, V9]`	**PASS**
T8 Tái lập	Chạy T1 hai lần cùng seed, `results/` giống từng byte	Mọi `V*.json` và `verdict.json` giống từng byte; T7 cũng được kiểm chạy hai lần giống nhau	**PASS**

6.3. Thảo luận các bất thường

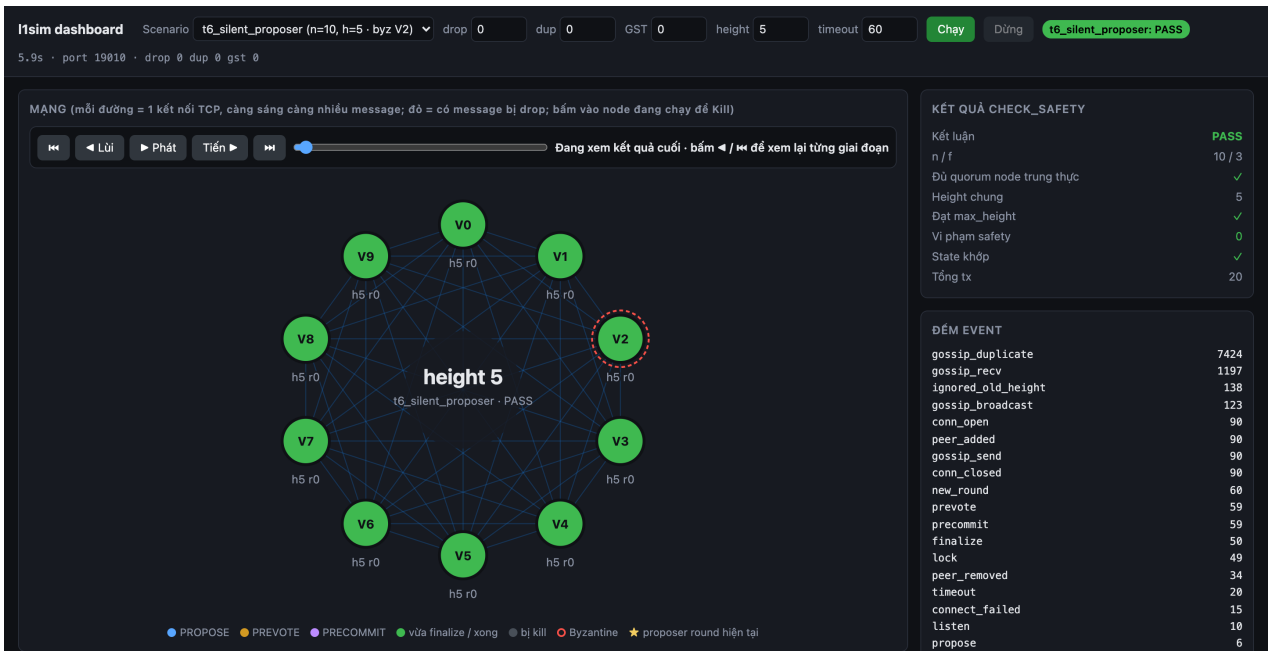
- **T5 commit height 1 ở round 1 và proposer là V2, không phải V1.** Round 0 mất quá nhiều message (drop 30% và trễ dài trước GST) nên không đạt polka; timeout đẩy sang round 1 với proposer $P(1,1) =$ V2. Từ height 2 trở đi (height $\geq$ GST) không còn drop nên mọi height lại commit ở round 0. Đây là đúng mô hình partial synchrony: trước GST chỉ cần safety.
- **T5 có 1 `round_skip`.** Một node bị chậm thấy $f + 1$ voter đã ở round cao hơn nên nhảy thẳng tới đó thay vì chờ hết timeout.
- **`commit_waiting_for_block` xuất hiện ở T2, T5, T6, T7.** Node nhận đủ $2f + 1$ precommit trước khi nhận body của block (vote đi qua gossip nhanh hơn chu trình header → request → body). Node chỉ chờ, không finalize mù; khi body tới thì finalize bình thường. Đây là hệ quả bình thường của header-before-body.
- **Số `lock` ít hơn số `finalize` (vd T2: 48/50, T5: 43/50).** Node nhận đủ precommit cho block trước khi tự thấy polka thì finalize luôn mà không qua bước lock, đúng luật 2.
- **T2 không có event `duplicate_ignored` của engine.** Gossip đã chặn bản trùng từ trước (event `gossip_duplicate`), nên bản trùng không tới VoteSet; dedup ở VoteSet vẫn được unit test kiểm riêng.
- **T7 height 6–9 không có tx.** 20 tx đã được commit hết ở height 1–5, nên các block sau rỗng; đây không phải lỗi.
- **Event log khác nhau giữa các lần chạy**, chỉ `results/` giống từng byte. Bộ đếm `time` của event là logical counter riêng từng node, còn thứ tự gossip phụ thuộc lịch của hệ điều hành. Đề chỉ yêu cầu results và state hash tái lập.
- **`connect_failed` ở mọi kịch bản** là lúc node khởi động trước peer của nó; StaticPeers quay số lại sau 0,1 s.

Hạn chế đã biết. (1) `PeerLimits` chưa gắn vào đường nhận nên chưa có chặn peer gửi quá nhanh ở runtime. (2) HELLO không có challenge–response: phát lại HELLO có thể làm kết nối chập chờn (ảnh hưởng liveness, không ảnh hưởng safety vì mọi vote/block có chữ ký riêng). (3) Chưa có block sync: node bị tắt không đuổi kịp chain khi bật lại. (4) Byzantine flood nhiều hash vote khác nhau buộc node lưu và chuyển tiếp tất cả; hành vi này nằm ngoài phạm vi lab. (5) `tx_root` là hash của danh sách tx, không phải Merkle root, nên không có bằng chứng tồn tại cho từng tx.

6.4. Demo trên giao diện web

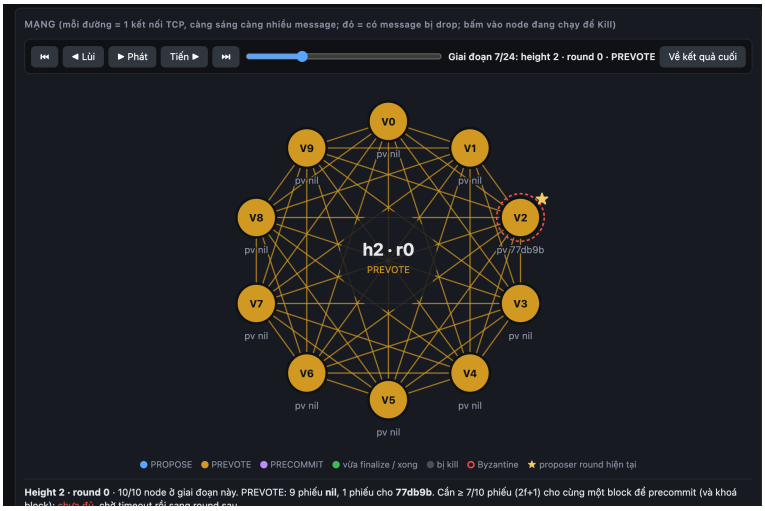
Để trình bày trực quan, nhóm viết thêm một dashboard web nhỏ (thư mục `dashboard/`, chỉ dùng thư viện chuẩn Python). Lệnh `python dashboard/server.py` mở trang tại `http://127.0.0.1:8080`. Dashboard không có logic blockchain riêng: khi bấm **Chạy**, server bật đúng 10 process `run_node` như `launch_local`, đọc dần các file `logs/events/<scenario>/V*.jsonl` mỗi 0,1 s để vẽ trạng thái, và khi mọi node dừng thì gọi `check_safety` để hiện verdict. Vì vậy những gì thấy trên màn hình chính là log của code thật. Người dùng chọn scenario, có thể sửa `drop`, `dup`, GST, `max_height`, kill một node đang chạy (tương đương Ctrl+C) và tải toàn bộ log dạng zip.

Hình 3 là kết quả khi nhóm demo T6 (proposer V2 im lặng). Vòng tròn có 10 node, mỗi đường là một kết nối TCP; V2 được đánh dấu Byzantine (viền đỏ nét đứt). Cả 10 node đạt height 5, panel `check_safety` cho PASS với 0 vi phạm safety, state khớp và 20 tx. Bảng chain bên dưới (không có trong hình) cho thấy height 2 do V3 commit ở round 1, khớp với bảng T6 ở mục 6.2.



Hình 3: Dashboard sau khi chạy T6: mạng 10 node và kết quả `check_safety` (PASS).

Sau khi chạy xong, thanh điều khiển (về đầu, **Lùi**, **Phát**, **Tiến**, về cuối cùng thanh kéo) cho phép xem lại từng giai đoạn PROPOSE → PREVOTE → PRECOMMIT → FINALIZE của mỗi height và round (T6 có 24 giai đoạn). Hình 4 là giai đoạn height 2, round 0, PREVOTE: V2 là proposer (dấu sao vàng) nhưng không gửi đề xuất, nên 9 node trung thực prevote *nil*; chỉ V2 prevote cho block của chính nó. Không block nào đạt 7/10 phiếu, các node chờ timeout rồi sang round 1, nơi V3 đề xuất và block được finalize. Nhóm dùng chế độ này khi demo để giải thích trực tiếp luật vote và cơ chế timeout trên log thật.



Hình 4: Xem lại giai đoạn height 2 · round 0 · PREVOTE của T6: 9 phiếu nil, 1 phiếu cho `77db9b`, chưa đủ $2f + 1$.

6.5. T1 với giao dịch nạp sẵn và giao dịch do người dùng gửi

Khi chạy T1 bình thường, các node *không* gửi giao dịch cho nhau. Trước khi bắt đầu đồng thuận, mỗi validator tự ký 20 giao dịch của scenario và nạp vào mempool của chính nó với `broadcast=False`. Vì cả 10 node đều có sẵn cùng tập này, gossip thêm chỉ tạo lưu lượng trùng. Để thử đường đi thật của một giao dịch, dashboard có chế độ **chạy liên tục**: mạng vẫn nạp sẵn 20 giao dịch scenario, chạy tới khi người dùng bấm dừng, và cho phép gửi giao dịch mới (ví dụ của Alice) qua `l1sim.client` tới một node bất kỳ. Ở chế độ này proposer giữ đề xuất round 0 khoảng 0,6 s, ít hơn `timeout_propose` = 1 s, để giao dịch kịp vào mempool. Bảng 2 so sánh hai cách.

Bảng 2: Giao dịch nạp sẵn và giao dịch do người dùng gửi trong T1.

	Nạp sẵn (T1 mặc định)	**Người dùng gửi (Alice qua client)**
Đường đi	Gọi hàm trong process, không qua mạng	TCP tới cổng của một node: kiểm HELLO có chữ ký, gửi frame `TX`, chờ phản hồi
Mempool	10 node có cùng mempool ngay từ height 1	Ban đầu chỉ node nhận có giao dịch; node đó gossip sang các node khác, và mỗi node kiểm tra lại trước khi nhận
Kiểm tra	`validate_pending_tx`: chữ ký → quyền sở hữu → nonce	Cùng hàm đó, chạy ở node nhận và ở từng node nhận qua gossip
Phản hồi	Không có	Client nhận `tx_id`, `accepted`, `reason`. `accepted` chỉ nghĩa là đã vào mempool, chưa phải đã commit
Event log	Không có event nhận giao dịch	`client_tx` ở node nhận, gossip loại TX; `tx_rejected` nếu bị từ chối
Height của giao dịch	Cố định: 20 giao dịch nằm ở height 1–5, mỗi block 4 giao dịch	Phụ thuộc thời điểm gửi: giao dịch chỉ vào block khi proposer của round đó đã có nó trong mempool
Tái lập (T8)	Cùng seed thì cùng block hash và state hash	Không đảm bảo, vì height và block hash phụ thuộc lúc người dùng gửi

Điều kiện PASS của T1 không đổi: mọi node trung thực finalize cùng block ở mọi height và có cùng state hash. Khi nhóm thử chế độ chạy liên tục, giao dịch hợp lệ của Alice được commit ở height sau cùng lúc trên cả 10 node. Bản sửa chữ ký bị từ chối với `bad_signature`, bản ký bằng domain `HEADER` bị từ chối với `wrong_domain`, và bản gửi lại giao dịch đã commit bị từ chối vì sai nonce. Giao dịch của người lạ (Mallory, không có trong genesis) bị từ chối với `bad_signature`. Kill một node trong lúc chạy thì giao dịch vẫn được commit, và `check_safety` khi dừng mạng vẫn cho PASS. Khác biệt duy nhất trong verdict là `total_txs` bằng 20 cộng số giao dịch của người dùng đã commit. Như vậy nạp sẵn dùng để chấm T1 và T8 vì kết quả tái lập được, còn đường client dùng để kiểm phần nhận giao dịch và các guard của nó, vốn không xuất hiện khi mọi giao dịch đều hợp lệ và có sẵn.

TÀI LIỆU THAM KHẢO

[1] Đề bài *Lab01 – Minimal Layer-1 Blockchain*, môn Blockchain, Trường ĐH Khoa học Tự nhiên, ĐHQG-HCM, 2026.

[2] E. Buchman, J. Kwon, Z. Milosevic. *The latest gossip on BFT consensus*. arXiv:1807.04938, 2018. `https://arxiv.org/abs/1807.04938`

[3] CometBFT (Tendermint) Consensus Specification. `https://docs.cometbft.com/v0.38/spec/consensus/`

[4] C. Dwork, N. Lynch, L. Stockmeyer. *Consensus in the presence of partial synchrony*. Journal of the ACM 35(2), 1988.

[5] M. Castro, B. Liskov. *Practical Byzantine Fault Tolerance*. OSDI 1999.

[6] S. Josefsson, I. Liusvaara. *RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA)*. IETF, 2017. `https://www.rfc-editor.org/rfc/rfc8032`

[7] A. Narayanan, J. Bonneau, E. Felten, A. Miller, S. Goldfeder. *Bitcoin and Cryptocurrency Technologies*. Princeton University Press, 2016.

[8] Python `cryptography` – Ed25519. `https://cryptography.io/en/latest/hazmat/primitives/asymmetric/ed25519/`

[9] Python `asyncio` – Streams. `https://docs.python.org/3/library/asyncio-stream.html`